

1 Go 语言实战

2 Stateflow 入门之旅

Stateflow 入门之旅

3 计算机视觉中的图像特征

3.1 填写函数的方式和原因

定义函数

函数定义由三个部分组成。

1. 函数声明以关键字 `function` 开头，定义调用函数的语法。
2. 函数体包含使用输入计算函数输出的命令。输出必须在函数体中定义。
3. `end` 关键字是函数的最后一行。

```
1 function out = myFunction(input)
2 % code that calculates out
3 end
```

调用优先级

在 MATLAB 中，设置了规则以解释任何命名项。这些规则称为函数优先顺序。可以使用以下顺序解决最常见的引用冲突：

变量

当前脚本中定义的函数

当前文件夹中的文件

MATLAB 搜索路径上的文件

3.2 计算机视觉入门之旅

计算机视觉入门之旅

课程简介：通过将典型工作流（检测跟踪）应用于海龟爬向大海的视频，学习计算机视觉的基础知识。您将了解特征在计算机视觉中的作用、如何标记数据、训练目标检测器以及在视频中跟踪野生动物。

使用目标检测跟踪海龟

在本课程中，您将学习如何使用目标检测来跟踪视频中的目标。课程将从导入视频数据开始，逐步讲解此检测跟踪工作流。

Import Video Data $\Rightarrow$ Label Ground Truth Data $\Rightarrow$ Train Detector $\Rightarrow$ Detect Objects in Each Frame $\Rightarrow$ Track Objects

3.2.1 创建视频读取器

可以使用 VideoReader 函数创建一个视频读取器，用于从文件中读取视频数据。

```
1 v = VideoReader("myVid.mp4")
```

创建视频读取器后，您可以使用 readFrame 函数从视频文件中提取帧。

```
1 v = VideoReader(filename)
2 fr = readFrame(v);
```

readFrame 的输出是图像，它是由像素值组成的数组。您可以在 readFrame 函数结尾使用分号，以避免 MATLAB 显示数百万个数字。

视频的一个帧就是一个图像。从视频文件中提取帧后，您可以像处理任何图像一样处理该帧。

```
1 imshow(frame)
2 %显示视频某一帧
```

在输出窗格的顶部，您可以看到 turtleVideo 的一些属性。例如，FrameRate 属性指示此视频的帧速率，即每秒显示的帧数

您可以用圆点表示法提取这些属性。

```
1 prop = obj.PropertyName
```

在上一个任务中，t 是 0.0333，即 $\frac{1}{frame\ rate}$ 秒。在第 1 行上创建视频读取器时，CurrentTime 属性的值为 0。

您可以使用 imshowpair 函数叠加两个图像，并以绿色和品红色显示它们的差异。

```
1 imshowpair(im1,im2)
```

读取特定帧

您可以反复调用 readFrame 以读取此视频的任一帧。但如何直接跳到第 90 帧呢？或如何回退到某个更早的帧？在本练习中，您将学习如何读取特定帧以及如何导航到视频中的任一帧。

在输出窗格的顶部，您可以在 turtleVideo 的输出中看到视频的总帧数和持续时间。变量 tMid 大约在视频的一半位置。

```
1 turtleVideo =
2   VideoReader - 属性:
3
4   常规属性:
5       Name: 'turtles.avi'
```

```

6         Path: '/users/mwa0000035781096/.training/.work'
7         Duration: 3.3333
8         CurrentTime: 0
9         NumFrames: 100
10
11    视频属性:
12         Width: 400
13         Height: 600
14         FrameRate: 30
15         BitsPerPixel: 24
16         VideoFormat: 'RGB24'

```

3.3 从参考图像中检测并提取特征

识别海龟的特征

要进行目标检测，您需要具有待检测目标的参考图像。参考图像应仅包含对检测该目标有用的特征。在此任务中，您将通过在视频帧中裁剪海龟周围的区域，来创建一个海龟的参考图像。

要指定要裁剪的矩形，需提供左上角的坐标以及矩形的宽度和高度（以像素为单位）。对于最小的 x 值和 y 值，您可以分别使用图像的列数和行数。

您可以通过 `imcrop` 函数指定要保留的矩形来裁剪图像。`imCropped = imcrop(im,rect);` 将矩形指定为一个四元素向量 `[xmin ymin width height]`。

```

1    rect = [xmin ymin width height];
2    imCropped = imcrop(im,rect);

```

您可以从矩阵或灰度图像中提取特征。由于 `turtle` 和 `frame` 是彩色图像，提取特征之前需将其转换为灰度图像。

您可以使用 `im2gray` 函数基于彩色图像创建灰度图像。

```

1    imGS = im2gray(im);

```

识别特征过程分为两部分：检测和提取。要检测图像中的类连通域特征，您可以使用 `detectSIFTFeatures` 函数。

输出是一系列感兴趣点的列表，其中可能包含尺度不变特征变换 (SIFT) 特征。

```

1    pts = detectSIFTFeatures(im)

```

现在您有了感兴趣点，可以使用它们来提取特征。感兴趣点标识了可查找特征的位置；一个点附近可能有多个特征，也可能一个特征也没有。

您可以对任何类型的特征使用 `extractFeatures` 函数。

第一个输出是特征矩阵。第二个输出是另一个 `SIFTpoints` 数组，其中包含这些特征的具体位置。

```
1 [feat,ftPts] = extractFeatures(im,pts)
```

您可以通过绘图对提取的特征可视化。大多数特征对应于海龟图像的某个区域。SIFT 特征是一种连通域特征。其他特征类型对应使用相同语法的函数。使用的特征类型取决于图像中的特征以及使用方法。

```
1 imshow(turtle)
2 hold on
3 plot(featurePoints)
4 hold off
```

您已从参考图像和原始视频帧中提取了特征，现在可以使用 `matchFeatures` 函数来匹配这些特征。

输出是一个包含匹配特征的索引的两列矩阵。`indexedPairs` 的第一列包含 `feat1` 的索引。第二列包含 `feat2` 的索引

```
1 indexedPairs = matchFeatures(feat1,feat2)
```

3.4 创建训练图像

脚本中的 `dir` 命令返回当前文件夹中 `.png` 文件的列表。您可以在输出窗格中看到此时没有这样的文件。

您可以使用 `objectDetectorTrainingData` 函数从视频文件创建训练图像。此函数将指定的每一帧写入一个图像文件，忽略不包含检测目标的帧。此语法使用 `SamplingFactor` 名称-值参量，每 `n` 帧提取一帧。

第一个输出是创建的图像的列表。第二个输出包含这些图像的标签。

```
1 [ims,labs] = objectDetectorTrainingData( ...
2     gTruth,SamplingFactor=n);
```

您可以通过在 `objectDetectorTrainingData` 函数中使用额外的名称-值参量来自定义图像文件的组织方式。例如，您可以标准化文件名或将图像写入特定文件夹。

这里，创建的图像的名称以 `basename` 开头，并保存在 `foldername` 文件夹中。

```
1 [ims,labs] = objectDetectorTrainingData( ...
2     gTruth,SamplingFactor=n, ...
3     NamePrefix="basename", ...
```

```
4 WriteLocation="foldername");
```

训练检测器时，您需要将图像列表和框标签合并为一个变量。您可以使用 `combine` 函数合并图像和标签。

```
1 imsWithLabels = combine(ims, labs);
```

3.5 训练 ACF 检测器

聚合通道特征 (ACF) 目标检测器使用图像的颜色通道和其他二维表示的特征来检测目标。

您可以使用 `trainACFObjectDetector` 函数创建一个使用真实值图像训练的 ACF 检测器。

```
1 d = trainACFObjectDetector(imsWithLabels)
```

您可以使用 `detect` 函数将 `detector` 应用于图像。[b,s] = `detect(detector,im)` 第一个输出是一个由边界框组成的矩阵，边界框标识 `im` 中检测到的目标。第二个输出是一个由置信度分数组成的矩阵，置信度分数指示每次检测的强度。

检测器发现了海龟，但也误将一块岩石标注为海龟。

具有带注解的边界框的视频帧，这些边界框围绕四只海龟和一块岩石

默认情况下，目标检测训练分为四个阶段。训练阶段越多，准确度越好，但需要的运行时间也越长。

您可以在 `trainACFObjectDetector` 函数中使用 `NumStages` 名称-值参量来设置阶段数。

```
1 detector = trainACFObjectDetector( ...  
2     imsWithLabels, ...  
3     NumStages=n)
```

3.6 对检测目标进行后处理

图像中每个带注解的检测目标都标注有其分数。分数越高，表示检测到的目标是海龟的置信度越高。

您可以通过设置分数阈值来删除低置信度的检测目标。用直方图可视化分数有助于确定阈值。

```
histogram(score,10)
```

根据直方图，您可以决定保留分数大于 50 的检测目标。但将阈值设置得太高也不合适，这会增加检测器对新帧的限制。我们先尝试保留分数大于 30 的检测目标。

3.7 对检测目标计数

现在，您可以将数值变量和用双引号引起来的文本串联起来创建一个字符串，以表示检测到的海龟数量。

```
n = 50;
```

```
s = "There are" + n + " states in the USA.";
```

您将在下一个任务中使用此字符串来注解图像。

您可以使用 `insertText` 函数通过文本来注解图像。

```
imNew = insertText(im,[xmin ymin],textStr);
```

第二个输入是一个由文本框左上角的 `x` 坐标和 `y` 坐标组成的向量。第三个输入是您要包含的字符串。

3.8 什么时候读取完所有帧？

在下一个练习中，您将学习如何处理视频的每帧。您如何知道何时停下来？例如，输出窗格显示当前时间是 0.9333。在本练习中，您将确定是否已读取所有帧。

您尝试读取下一帧之前，可以使用 `hasFrame` 函数确定是否还有任何剩余帧。

```
yn = hasFrame(v)
```

输出是一个逻辑值，如果视频读取器中仍有待读取的帧，其值为 `true`；如果读取结束，则为 `false`。

3.9 对每帧应用检测算法

在本练习中，您将海龟检测算法应用于视频第一秒内的每一帧。

在工作区中，`detector` 变量已基于更多数据进行了训练。此检测器比您之前创建的检测器更准确，但花费的训练时间更长。

您可以使用 `while` 循环重复代码，直到满足某个条件。

此处，`condition` 为 `true` 或 `false`，只要为 `true`，循环内的代码就会重复。在此任务中，您需要重复执行代码，直到已遍历所有帧（可通过 `hasFrame` 函数确定是否已遍历所有帧）。

```
1 while condition
2     % code that does stuff
3 end
```

3.10 初始化卡尔曼滤波

在下一课中，您将使用卡尔曼滤波器跟踪此视频中的海龟。您通过跟踪能够知道海龟的位置，虽然它在故障期间检测不到。

要设置卡尔曼滤波器，您需要对目标的运动做一些假设。您觉得它是均匀加速运动（就像下落的物体）还是匀速运动？视频中的海龟在剪辑期间似乎以相同的速度逐帧移动。

您使用的运动模型会影响需要指定的关于运动和测量噪声的其他参数。您在前五个任务中定义参数，然后在任务 6 中使用它们初始化卡尔曼滤波器。

初始化卡尔曼滤波器所需的第二个假设是初始位置，它是根据边界框计算的。

`initialLoc = centroid`

课程将测量噪声定义为方差。质心出现了约 10 个像素的位移。因此，您可以对测量噪声参数使用

$$10^2 = 100$$

要初始化用于跟踪的卡尔曼滤波器，请使用 `configureKalmanFilter` 来指定您将跟踪的运动类型的假设。

```
1 kalmanFilter = configureKalmanFilter(MotionModel,...  
2     InitialLocation,...  
3     InitialEstimateError,...  
4     MotionNoise,...  
5     MeasurementNoise)
```

3.11 MATLAB 数据之旅

学习内容

在本课程中，您将学习：

以浮点和整数数据类型存储数值数据。

通过了解数值数据的表示方式避免错误。

将文本数据存储为字符数组和字符串数组。

在不同的文本表示形式之间进行转换。

创建并修改分类数组。

将不同种类的数据存储为合适的 MATLAB 数据类型。

数值数据类型

由于数值数据是 MATLAB 中最常用的数据，因此本课程首先深入探讨各种可用的数值数据类型，包括它们存储值的方式和常见用例。

您可能已经注意到，在 MATLAB 的变量中存储数值时，无需指定要使用的数据类型。

```
1 >> x = 3.14
2
3 x =
    3.1400
```

如果您想查看某个变量的数据类型，可以使用 `class` 函数。

```
1 >> class(x)
2 ans =
3 'double'
```

默认的数值数据类型是 `double`，也就是双精度浮点值。浮点值在计算中使用广泛；它们支持快速计算，并且具有足够的精度，适用于大多数实际应用。但是，有限精度和浮点算术有时可能会导致意外的结果。

浮点数和有限精度

计算机用于存储数值的内存是有限的，因此其使用了数值的有限精度表示。在 MATLAB 中，数值变量默认以 64 位双精度浮点值（即 `double`）类型存储。

任何进制中都存在无法用有限精度精确表示的数值。例如，十进制中无法精确表示 $1/3$ ；无论写出多少位，它永远不会是精确的。同样，二进制中无法精确表示 $1/10$ (0.1)。

使用双精度，您可以得到非常接近 0.1 的结果（双精度提供大约 16 位有效数字）。大多数情况下，非常接近就已经足够了。然而，有时这些微小的差异是有影响的。了解浮点二进制格式中不存在 0.1 的精确表示可以帮助避免混淆、错误，并减少技术支持请求。

您可以使用 `single` 函数创建单精度变量，或将现有的数值变量转换为单精度。

```
x = single(3.14)
```

整数表示

表 1: 无符号

类型	内存（字节）	范围
uint8	1	0 - 255
uint16	2	0 - 65535
uint32	4	0 - 4294967295
uint64	8	0 - 18446744073709551615

根据以下特征，整数变量分为几种不同的类型：

表 2: 有符号

类型	内存（字节）	范围
int8	1	-128 -127
int16	2	-32768 -32767
int32	4	-2147483648 -2147483647
int64	8	-9223372036854775808 -9223372036854775807

表示整数变量所用内存量。

有符号还是无符号。

一个 n 位整数可以表示 2^n 个值。无符号值不能为负值，因此从 0 开始。有符号值可以是正值或负值，并以 0 为中心。

例如，8 位整数可以表示 256 (28) 个值。无符号 8 位整数（“uint8”）的取值范围是 0 到 255，而有符号 8 位整数（“int8”）的取值范围是 -128 到 127。

比较两个数组

您可以使用 `==` 运算符比较数组的元素。您也可以使用 `isequal` 函数比较数组是否等效。

表 3: 比较两个数组

<code>x == y</code>	<code>isequal(x,y)</code>
逐元素比较	整个变量比较
结果是逻辑数组，其大小取决于 <code>x</code> 和 <code>y</code> 的大小	结果是逻辑标量
比较大小不兼容的数组会产生错误	如果 <code>x</code> 和 <code>y</code> 具有相同的大小和值，则结果为 <code>true</code>

请注意，`==` 和 `isequal` 都用于测试是否完全相等。在浮点算术中，更适合的测试方法是验证您应用中的两个值是否“足够接近”：

$$\|x-y\| < \epsilon$$

其中 ϵ 是一个很小的容差值。

在接下来的任务中，您将探究这些不同的数组比较方法。

数值数据类型摘要-数值类型

double - 浮点数，8 字节，精度约为 16 位数字

single - 浮点数，4 字节，精度约为 8 位数字

intn - 有符号整数（请参阅下表）

uintn - 无符号整数（请参阅下表）

使用单引号创建的文本是字符的行向量。与任何数组一样，您可以使用方括号将行向量串联在一起。结果为另一个行向量。

```
1 greet = 'Hello';
2 space = ' ';
3 whom = 'world';
4 txt = [greet space whom]
5
6 txt =
7     'Hello world'
```

whos

Name	Size	Bytes	Class	Attributes
------	------	-------	-------	------------

greet	1x5	10	char	
-------	-----	----	------	--

space	1x1	2	char	
-------	-----	---	------	--

txt	1x11	22	char	
-----	------	----	------	--

whom	1x5	10	char	
------	-----	----	------	--

正如您所看到的，对字符串使用方括号会创建字符串数组。而要将字符串合并成标量字符串，可以使用加号运算符 (+)。结果为另一个标量字符串。

```
1 greet = "Hello";
2 space = " ";
3 whom = "world";
4 txt = greet + space + whom
5
6 txt =
7     "Hello world"
```

whos

Name	Size	Bytes	Class	Attributes
------	------	-------	-------	------------

greet	1x1	150	string	
-------	-----	-----	--------	--

space	1x1	150	string	
-------	-----	-----	--------	--

txt	1x1	166	string	
-----	-----	-----	--------	--

whom	1x1	150	string	
------	-----	-----	--------	--

字符串串联的一个常见用途是以编程方式组合各个部分构建字符串，例如使用文件名的各个组成部分构建完整的文件名。

```

1 f = "myfile";
2 ext = ".txt";
3 fullf = f + ext

```

不同的操作系统使用不同的符号（“/”或“\”）来分隔文件名和文件夹名称。良好的编程习惯是使用 filesep 而非硬编码指定分隔符。

file = "folder" + filesep + "file"

数值 filenum 已自动转换为文本。这是一种生成遵循某种模式的多个文件名的有效方法。

```

1 for k = 1:5
2     thisfile = path + filesep + ...
3         file + k + extension
4 end

```

要创建元胞数组，您可以使用花括号 ({ }) 将元素串联在一起，就像使用方括号创建任何其他类型的数组一样。

用空格或逗号水平分隔元素，用分号垂直分隔元素。例如，以下代码创建一个 2×2 元胞数组：

```

1 c = {1 'hello'; zeros(7,3) [42 666]}
2
3 c =
4     2×2 cell array
5     {[          1]}    {'hello' }
6     {7×3 double}    {[42 666]}

```

c 的四个元胞包含不同大小和类型的数据。用于存储文本的元胞数组通常为单行或单列，其中每个元胞都包含一个字符向量。

```

1 c = {'apple','banana','cherry'}
2
3 c =
4     1×3 cell array
5     {'apple'}    {'banana'}    {'cherry'}

```

您可以使用圆括号对元胞数组进行索引，就像对任何其他数组进行索引一样。

cfirst3 = c(1:3)

通常进行索引会提取出一个与原始数组类型相同的较小数组。

数组索引运算 `greet(2)` 会生成一个包含字符向量 'Bonjour le monde' 的 1×1 元胞数组。如何访问该字符向量（即元胞内容）以便进一步处理它？

您可以使用花括号 (`{ }`) 对元胞的内容进行索引。

```
c2contents = c2
```

与常规数组索引一样，您还可以组合使用元胞索引和赋值来修改元胞的内容。

```
c2 = 'new content'
```

使用数组索引时，为不存在的元素赋值会导致数组大小更改。您可以将相同的原则应用于元胞索引。

```
greet5 = ' ', ' '
```

此命令用于将字符向量赋给元胞的内容。

如果您尝试将字符向量赋给元胞，会出现错误。

```
greet(6) = ' こんにちは世界' % not allowed
```

请记住：对于元胞数组，常规数组索引 (`()`) 引用的是元胞，而元胞索引 (`{ }`) 引用的是元胞的内容。您可以记忆英文口诀 *curly for content*（花括号引用内容）。

您可以赋值给元胞，但前提是所赋的值本身就是元胞。

```
greet(6) = {' こんにちは世界'}
```

请注意，与 `fruitstring` 相比，`fruitcell` 需要的内存要大得多。

元胞数组是通用容器变量。每个元胞（指元胞容器本身）需要 104 字节的内存。这是除元胞内容所需内存之外需要的内存。因此，除了单个字符所需的 1,146 字节之外，`fruitcell` 还需要 10,400 字节的内存。

通常在处理文本数据时，会将其存储为字符串数组，而不是字符向量元胞数组。

MATLAB 提供许多用于执行文本操作的函数，例如在文本数组中查找、提取和替换匹配的文本模式。

文本输入可以采用任何形式（`char`、`string` 或 `char` 向量元胞数组）。对于将文本作为输出返回的函数，输出类型与输入类型匹配。

例如，以下代码提取每个文本元素中第一个空格字符之前的部分。

```
1 greet = {'Hello world', 'Bonjour le monde', 'Kia ora te ao', 'Halló  
heimur'};  
2 firstword = extractBefore(greet, " ")  
3  
4 firstword =  
5     1×4 cell array  
6     {'Hello'}     {'Bonjour'}     {'Kia'}     {'Halló'}
```

请注意，`extractBefore` 的第一个输入是字符向量元胞数组，第二个输入是字符串。它的输出类型与第一个输入的类型匹配 - 此处四个文本片段已指定为字符向量元胞数

组，因此返回的四个子字符串为相同的格式。如果将这些文本指定为字符串数组，则输出也是字符串数组。

```
1 greet = ["Hello world","Bonjour le monde","Kia ora te ao","Halló  
    heimur"];  
2 firstword = extractBefore(greet," ")  
3  
4 firstword =  
5     1×4 string array  
6     "Hello"      "Bonjour"      "Kia"      "Halló"
```

变量 ob 包含 15 个碰撞测试的目标列表。

我们可以看到，向量 ob 包含了一些无意义的多余文本，导致数据杂乱。例如，每个测试目标前都带有“objective”一词，其实没有必要。

您可以使用 erase 函数从字符串数组中删除这一文本模式。newstr = erase(str,pattern)

清除某些短语后，可能会留下前导或尾随空白。您可以使用 strtrim 函数将其删除。

newstr = strtrim(str)

这里许多目标数据都与乘客约束信息有关。要查找包含“restraint”一词的所有测试目标，您可以使用 contains 函数。idx = contains(str,pattern,IgnoreCase=true)

输出 idx 是一个逻辑数组，用于识别 str 中哪些元素包含 pattern（不区分大小写）。

然后您可以使用该逻辑数组对原始数组进行索引，以提取已识别的元素。mystr = str(idx)

文本数据类型摘要

文本变量：字符和字符串

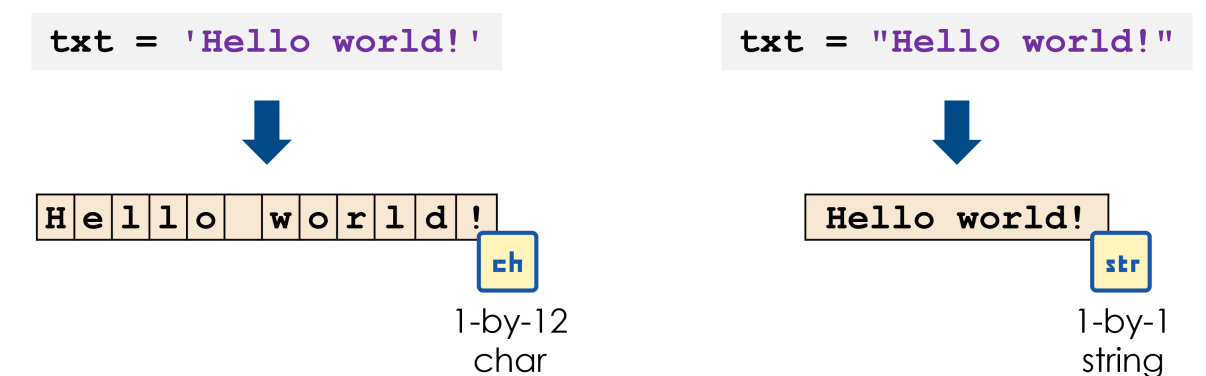


图 1: 字符和字符串

表 4: 字符和字符串

'char'	"string"
使用单引号会创建 char 数组 标量 char 是单个字符 char 数组是字符数组 要存储多个不同长度的字符行向量，请使用元胞数组	使用双引号会创建（标量）string 标量 string 是任意字符数的单个文本单元 string 数组是字符串数组

字符向量元胞数组

通过使用花括号 ({ }) 串联元素来创建元胞数组。

```
1 goodbyes = {'Arrivederci', 'さようなら', 'Auf Wiedersehen',  
2            '}'  
3  
4 goodbyes =  
5     1×4 cell array  
        {'Arrivederci'}      {'さようなら'}      {'Auf Wiedersehen'}      {  
            '}'
```

使用圆括号对元胞数组进行索引会返回另一个元胞数组。

```
1 goodbyes(2:3)
2
3 ans =
4     1×2 cell array
5     {'さようなら'} {'Auf Wiedersehen'}
```

使用花括号进行索引来提取元胞的内容。

```
1      goodbyes{4}
2
3      ans =
4      1      1      1      1
```

表示离散类别

有时候，您的文本数据由表示有限可能性集的文本标签组成。categorical 数据类型正是为这种情况而设计的。

MATLAB 有许多用于处理数据类别的函数。此外，string 数组可能比 categorical 数组占用更多内存。

因此，如果您的文本描述的是一个组或类别，使用 categorical 数据类型往往是最好的选择。

有些函数旨在作用于标签或类别，而不是纯文本。例如，您可以创建直方图来可视化类别中的元素数。但一般来说，文本直方图可能意义不大。

您可以使用 `categorical` 函数将 `string` 数组转换为 `categorical` 数组。`catArray = categorical(stringArray)`

分类数组比字符串数组占用的内存更少，因为它们可以更高效地存储重复的字符串元素。

您可以使用 `categories` 函数查看分类数组中表示的类别。

```
1 Create and Operate on Categoricals
2 Instructions are in the task pane to the left. Complete and submit
  each task one at a time.
3
4 This code sets up the activity.
5 x = ["C","B","C","A","B","A","C"]
6 This line of code would produce an error.
7 % histogram(x)
8
9 Task 1
10 y = categorical(x)
11
12 Task 2
13 whos x y
14
15 Task 3
16 cats = categories(y)
17
18 Task 4
19
20 Further Practice
```

前面提到，您可以使用点索引来访问表中的变量。`tbl.VariableName`

要将表变量转换为分类变量，您可以使用 `categorical` 函数并将输出重新赋给该表变量。

提示

使用 `categorical` 函数。

```
tableName.varName = categorical(tableName.varName)
```

任务

使用 `nnz` 函数计算 `players` 的 `Position` 变量中的“striker”值的数量。并将结果保存

在名为 ns 的变量中。

提示

使用 nnz 函数，其中 valueOfInterest 为”striker”。

```
numVals = nnz(tbl.VarName == "valueOfInterest")
```

您可以使用 summary 函数查看类别名称以及每个类别中的元素数。

```
summary(players.Team)
```

Arsenal 3

Crystal Palace 1

Everton 1

Liverpool 1

Manchester City 1

Manchester United 1

Tottenham Hotspur 2

提示

使用 summary 函数。

```
summary(tableName.VariableName);
```

什么是函数句柄？

函数句柄是一个变量，存储对函数的引用。

为什么需要使用这样的变量？函数句柄通常用作将一个函数传递给另一个函数的一种方式。

函数句柄的创建和使用

创建已定义函数的函数句柄

使用 @ 符号创建引用现有函数的函数句柄。您可以引用 MATLAB 函数、局部函数或函数文件中定义的函数。



图 2: 创建已定义函数的函数句柄

使用函数句柄

定义完函数后，可以使用输入调用函数句柄，也可以将函数句柄作为输入传递给另一个函数。

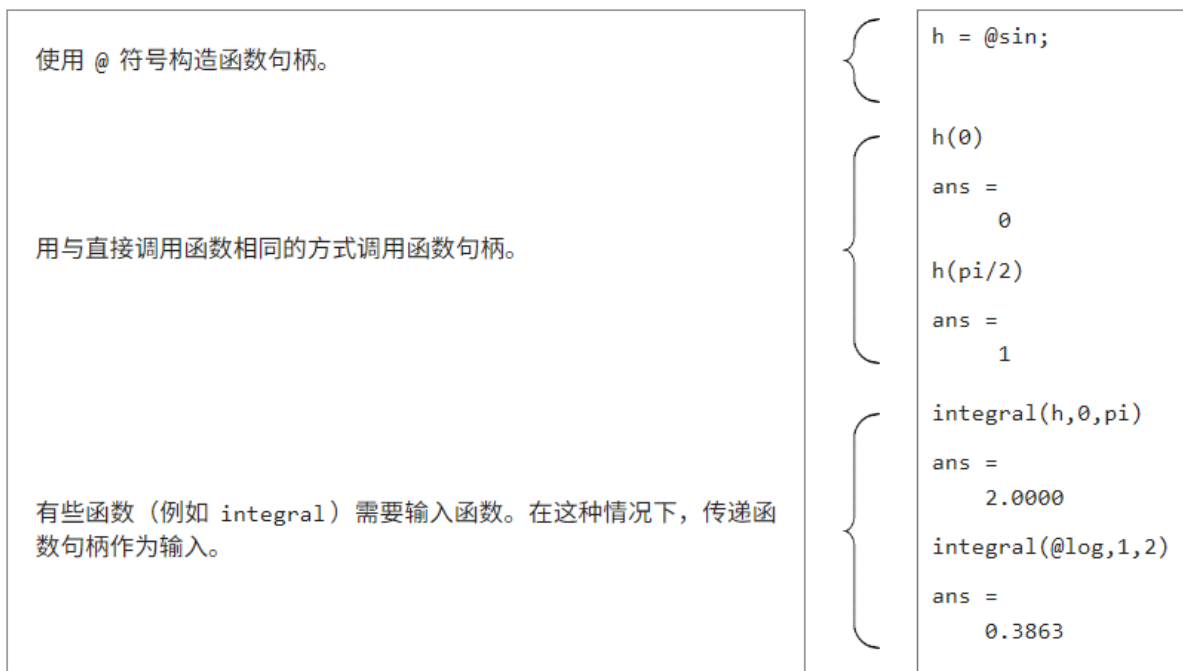


图 3: 创建已定义函数的函数句柄

匿名函数

匿名函数是在代码中直接定义的函数，而不是使用 `function` 关键字创建的单独的命名函数。

您可以使用以下语法创建匿名函数的函数句柄：

`function_handle = @(in1,in2,...)definition`

与以往一样，@ 符号用于定义函数句柄。您无需指定函数的名称，但是要在圆括号中列出输入参量，然后给出函数的定义。因此，匿名函数必须在一行代码中完成定义。

例如，`f = @(x,a,b) sin(a*x+b)` 将创建一个名为 `f` 的函数句柄，该函数句柄定义一个包含三个变量 `x`、`a` 和 `b` 的函数。该函数返回值 `sin(a*x+b)`。

您可以将此语法视为用于定义函数文件的语法的缩写形式。

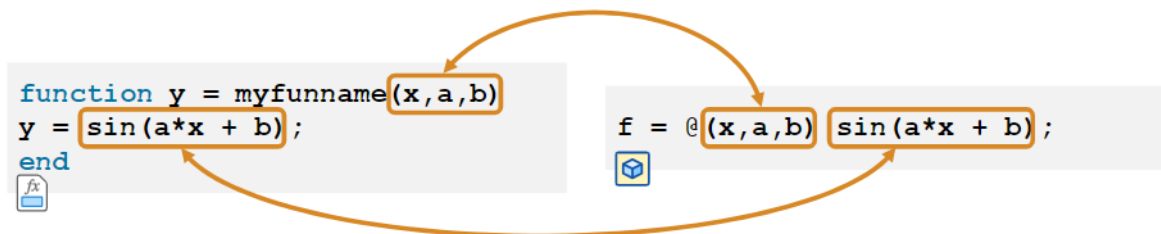


图 4



图 5

什么是对象？

对象是专门用于存储特定信息集合的特殊用途变量。对象具有与之结合使用的函数（也称为方法）。

MATLAB 为各种专用任务提供对象。对象广泛用于各种 MATLAB 工具箱。

例如，您可以在 MATLAB 中将多边形表示为 `polyshape` 变量。您可以使用 `plot` 函数可视化多边形，并使用 `area` 函数计算其面积。如果使用 `nearestvertex` 函数，则可以找到最接近给定点的多边形顶点。

```
1 p = polyshape(x,y);  
2 plot(p)  
3 A = area(p);  
4 id = nearestvertex(p,0.5,3.14);
```

当然，还有一个用于数值数组的 `plot` 函数，但用于多边形的 `plot` 函数是专门为可视化多边形而设计的。`nearestvertex` 函数是多边形所独有的，因此仅适用于多边形变量。

术语

对象和方法等术语在编程中具有特定含义。在某些语言中，对象与其他种类的变量之间的区别可能很重要。然而，在 MATLAB 中，大多数情况下您可以将“对象”视为等同于“（专用类型的）变量”，将“方法”视为等同于“函数”。

专用数据类型摘要函数句柄

函数句柄是一个变量，存储对函数的引用，通常用于将一个函数传递给另一个函数。

定义函数句柄

`f = @funname` `f = @(inputs) definition`

匿名函数可以在其定义中使用工作区变量。

`a = 2; b = pi/4; f = @(x) sin(a*x + b);`

在创建匿名函数时工作区变量的值已嵌入在匿名函数的定义中。之后即使更改变量的值，也不会影响该匿名函数。

使用函数句柄

您可以通过计算函数句柄来代替函数调用，也可以使用函数句柄作为另一个函数的输入

```
1  f = @sin;
2  y = f(pi/4)
3
4  y =
5      0.7071
6
7
8  z = fzero(f,4)
9
10 z =
11     3.1416
```

对象

对象是专用数据类型的变量。方法是用于处理该类对象的函数。对象通常具有可以使用圆点表示法访问的属性。

```
1  p = polyshape(x,y)      % Make a polyshape object
2  v = p.Vertices          % Extract the Vertices property
3  [x0,y0] = centroid(p)  % Call the centroid method
```

字典

字典是一种数据容器，通过它可以将元素映射到键。您可用键进行索引，而无需使用数值索引。键可以是任何数据类型。这意味着您可以按名称查找数据，例如：

```
1  total_pets = d("cat") + d("dog") + d("elephant")
```

表

表专为由多个相互关联的变量组成的表格数据而设计，这些变量具有以下特点：

每个变量都是一种数据类型

不同变量可以是不同数据类型

每个变量具有相同的行数

每一行的每个变量都对应相同的观测值，因此尽管表的行可以重新排列，但是它们必须保持为一个整体

元胞和结构体

在本课程中，您使用了元胞数组来存储不同长度的字符向量。但是，元胞数组的用途远不止于此；它们可以存储完全异构的数据。也就是说，每个元素（元胞）都可以包含任何大小和类型的数据。

结构体也是完全异构数据的容器。元胞数组使用数值索引来标识元素，而结构体使用命名索引（类似于表）。

当数据具有不规则结构（通常是不同大小的元素，这意味着数据不能以任何其他数据类型存储）时，元胞和结构体非常有用。您可以在《元胞和结构体》课程中了解有关使用元胞数组和结构体的详细信息。

日期和时间变量

MATLAB 提供三种与日期和时间相关的数据类型。这些数据类型协同工作，让您很好地对基于时间的数据执行操作和计算，例如按日期排序、计算时间点之间的持续时间、添加不同单位的持续时间等。

`datetime` 类型存储特定时间点，例如 2005 年 10 月 21 日。`duration` 和 `calendarDuration` 类型存储时间段 - `duration` 表示与上下文无关的时间长度，例如小时；`calendarDuration` 表示依赖于上下文的时间长度，例如月（可以是 28、29、30 或 31 天）。

您可以在课程《操作日期和时间》中了解有关用于处理基于时间的数据的 MATLAB 数据类型的详细信息。

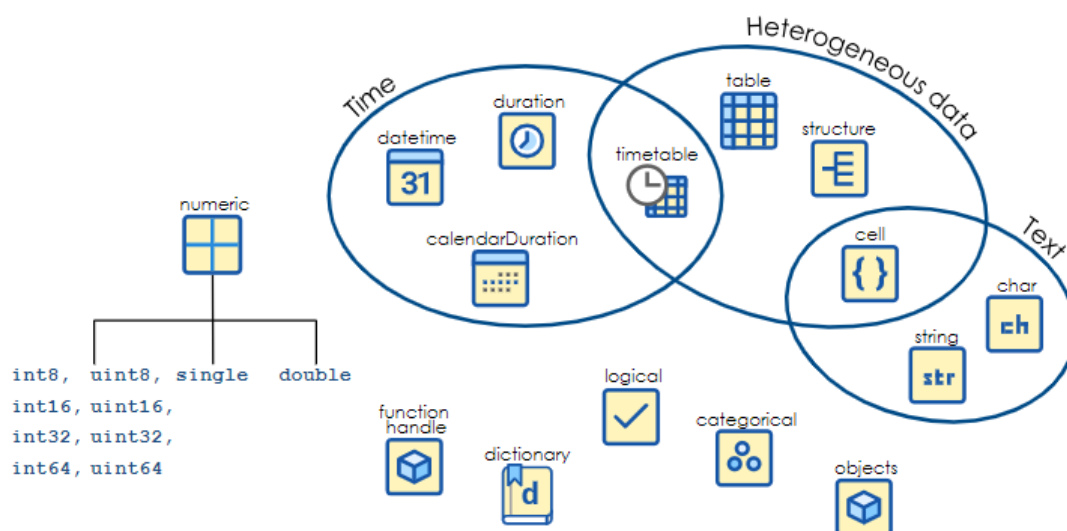


图 6: 其余数据类型摘要

Coding Practices for Efficiency and Performance

A subset of vectorized functions and operations is provided below. You should look for opportunities to vectorize your code and consult MATLAB documentation for possible solutions.

Element-wise Operations		
+	-	
.*	./	.^
Mathematical Functions		
sin	exp	sqrt
abs	round	etc.
Statistical Functions		
sum	mean	max
std	prod	diff
cumsum	cumprod	etc.
Set Operations		
union	intersection	unique
setdiff	setxor	ismember
String Operations		
strcmp	strncmp	regexp
strcat	strvcat	cellstr
deblank	lower	etc.
Logical Functions		
is*	any	all
nnz	find	

图 7

3.12 MATLAB Coding Practices for Efficiency and Performance

Specialized Data Types MATLAB has data types that are designed to be used with data having specific characteristics. These data types can significantly reduce the memory consumption.

Datetime, categorical, and text arrays can be stored as cell arrays of character vectors, but by using a specialized data type, you can reduce memory usage and get faster performance from functions designed to work on these specific data types.

Cell array of character vectors 912 Bytes	String array 474 Bytes ★	Categorical array 497 Bytes
<pre>names = {'Rafael' 'Roger'... 'Rafael' 'Novak' 'Roger'... 'Andy' 'Novak'};</pre>	<pre>names = ["Rafael" "Roger"... "Rafael" "Novak" "Roger"... "Andy" "Novak"];</pre>	<pre>names = categorical({'Rafael',... 'Roger' 'Rafael' 'Novak'... 'Roger' 'Andy' 'Novak'});</pre>

Cell array of character vectors 1560 Bytes <pre>names = {'Roger' 'Roger'... 'Roger' 'Roger' 'Rafael'... 'Roger' 'Rafael' 'Novak'... 'Roger' 'Andy' 'Novak'... 'Andy'};</pre>	String array 744 Bytes <pre>names = string({'Roger'... 'Roger' 'Roger' 'Rafael'... 'Rafael' 'Roger' 'Rafael'... 'Novak' 'Roger' 'Andy'... 'Novak' 'Andy'});</pre>	Categorical array 502 Bytes ★ <pre>names = categorical({'Roger'... 'Roger' 'Roger' 'Rafael'... 'Rafael' 'Roger' 'Rafael'... 'Novak' 'Roger' 'Andy'... 'Novak' 'Andy'});</pre>
--	---	---

Cell array containing character vectors 688 Bytes <pre>dates = {'1/28/2015' '1/29/2015'... '1/30/2015' '1/31/2015'... '2/1/2015'};</pre>	String array 446 Bytes <pre>dates = string({'1/28/2015'... '1/29/2015' '1/30/2015'... '1/31/2015' '2/1/2015'});</pre>	Datetime array 41 Bytes ★ <pre>dates = datetime({'1/28/2015'... '1/29/2015' '1/30/2015'... '1/31/2015' '2/1/2015'});</pre>
--	---	--

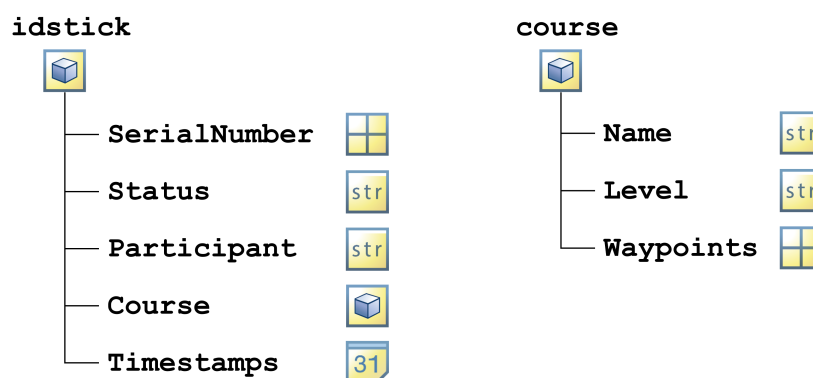
3.13 面向对象的编程之旅

面向对象的编程之旅

处理最终产品

在前面几个练习中，您将使用预先创建的自定义对象版本，在后面的课程中，您将自己创建这些自定义对象。

为了模拟定向运动赛事，您需使用两个自定义对象 - 一个用于表示参赛人员可以行进的路线，另一个用于表示参赛人员携带的 ID 棒以跟踪其进度。



您将在练习中熟悉这些对象 - 它们包含哪些信息，它们有哪些功能。在其余的课程部分，您将学习如何从头开始创建它们。您还会接触一些在面向对象的编程 (OOP) 中使用的术语。如果您无法立刻完全理解，也不要担心 - 您会在其余课程部分详细地学习，它们并不像听起来那样复杂！

如果您想先睹为快，可以在编辑器中打开定义自定义（course 和 idstick）对象的文件。但其中的大量代码可能让您望而生畏。课程结束时，您会完全了解其所有功能。

定向运动路线由名称、难度级别和一个路点列表（其形式为数字组成的数组）来定义。通过使用这三个输入调用 course 函数，您可以创建一个新的 course 对象（变量）。

```
1 c = course("Name","Level",waypoints)
```

在许多说英语的国家/地区，难度级别以颜色来表示，例如“白色”表示最简单的路线，“橙色”表示中级难度，“红色”表示高级难度等。

一旦参赛人员使用 ID 棒来注册以沿路线行进，ID 棒就可以保留大量信息，但在第一次创建 ID 棒时，它基本上是空白的。您可以通过一个表示 ID 棒的序列号的输入调用 idstick 函数来创建一个表示 ID 棒的对象。

```
1 id = idstick(serialnumber)
```

您刚刚创建的变量是自定义类 course 和 idstick 的对象。

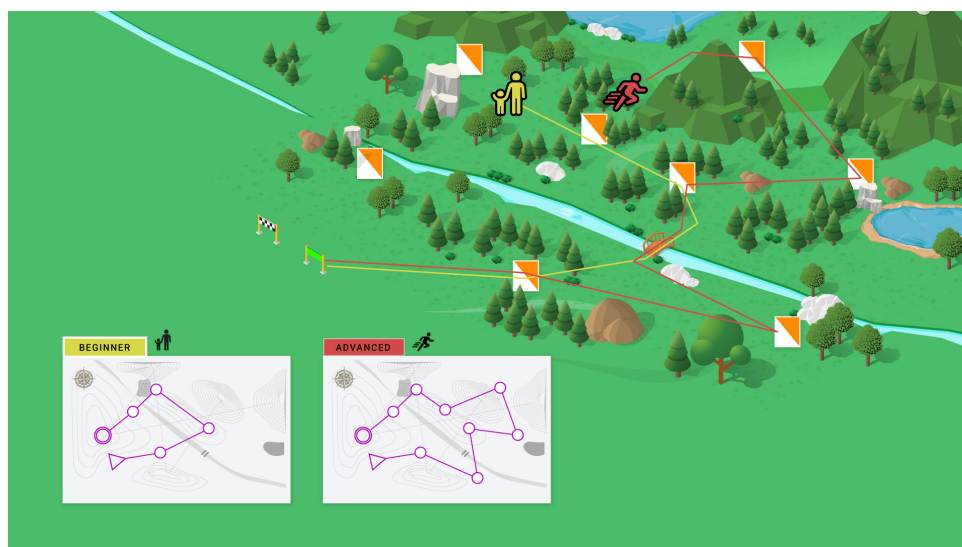
您对自定义对象 c 和 id 了解多少？它们保留哪些信息？您能用它们做什么？

您可以使用 properties 和 methods 函数来查看工作区中对象的 properties（包含的信息）和 methods（使用的函数）的名称。

```
1 properties(x)
2 methods(x)
```

ID 棒、路线和注册

在定向运动赛事中，参赛人员可以选择自己行进的路线。他们需要使用 ID 棒来向特定路线注册。在注册过程中，ID 棒会加载路线信息，因为参赛人员在沿途的路点签到时需要 ID 棒提供这些信息。



ID 棒对象有一个名为 `register` 的方法。它模拟参赛人员用其 ID 棒进行路线注册。

```
1 id = register(id,"Name",c)
```

输入是：ID 棒对象、一个表示参赛人员姓名的字符串、一个表示参赛人员注册的路线的路线对象。

请注意，ID 棒发出 beep 声表示注册成功。这是一种模拟物理 ID 棒向参赛人员提供反馈的简单方式。如果省略分号，还会显示 ID 棒现已注册。

ID 棒对象还有一个方法用于模拟参赛人员在路点签到。

```
id = checkWaypoint(id,waypt)
```

其中 `waypt` 是路点的编号。

成功签到后，ID 棒还会发出 beep 声。第一个路点是路线的起点。在此处签到还会将状态从 `ready to run` 更改为 `running`。

欢迎使用 MATLAB，专为处理数组而打造的产品

MATLAB 中的所有变量，甚至包括自定义对象，均为数组。与所有其他类型的变量一样，单个（标量）对象是一个 1×1 数组。

使用对象数组是否合适？对象向量的行为应是怎样的？这些设计决策取决于您的对象所表示的内容。

以定向运动为例，您可以在一次赛事中有多条路线和多个 ID 棒供参赛人员使用，因此适合使用 `course` 和 `distick` 对象数组。

一个定向运动赛事中有多个参赛人员和多条路线。为了与 MATLAB 的向量化特性保持一致，最好将赛事表示为一个由路线对象组成的数组和一个由 ID 棒组成的数组。

当前 `c` 是一个由标量（即单个元素）表示的路线对象。

每个 `idstick` 对象都有一个序列号。但是，使用序列号数组调用 `idstick` 函数将创建一个由 ID 棒组成的数组（每个元素对应一个序列号）。

```
1 id = idstick([123456, 567890, 987654])
```

虽然创建 ID 棒和路线组成的向量是合理的，但参赛人员一次只能使用一个 ID 棒沿一条路线行进。因此，`register` 函数的所有输入都需要为标量。但是，您可以对由对象组成的数组 (`obj(k)`) 的单个元素进行索引，就像任何其他类型的数组一样。

前面提到，`register` 函数使用如下语法：

```
id = register(id,"Name",c)
```

多个参赛人员可以同时注册同一条路线并沿该路线行进，只要他们每个人都使用自己的 ID 棒即可。

要创建名为 `classname` 的新类，请创建一个名为 `classname.m` 的（纯文本）代码文件，其中包含类定义块

```
classdef classname
```

end

如果您有类定义文件 `classname.m`，命令

```
obj = classname
```

将创建一个新变量（OOP 术语为对象）`obj`，其类型为 `classname`。

请注意，在整个课程中，您将编辑类定义文件以及使用这些类的脚本。使用编辑器顶部的文件选项卡可在打开的文件之间切换。

通过在类定义的 `properties` 块内列出属性名称来定义类的属性

```
1 classdef classname
2     properties
3         Propname1
4         Propname2
5         ...
6     end
7 end
```

在编辑代码文件时，MATLAB 编辑器使用智能缩进来提高代码的可读性。但是，编辑过程中您可能会发现代码不对齐。在这种情况下，您可以选择代码并使用 `Ctrl+I`（在 macOS 系统上为 `Command+I`）来重新应用缩进。

您可以使用 `object.Property` 索引访问存储在自定义对象的属性中的值，就像在 MATLAB 中访问任何其他对象一样。

```
obj = classname;
```

```
propvalue = obj.PropertyName
```

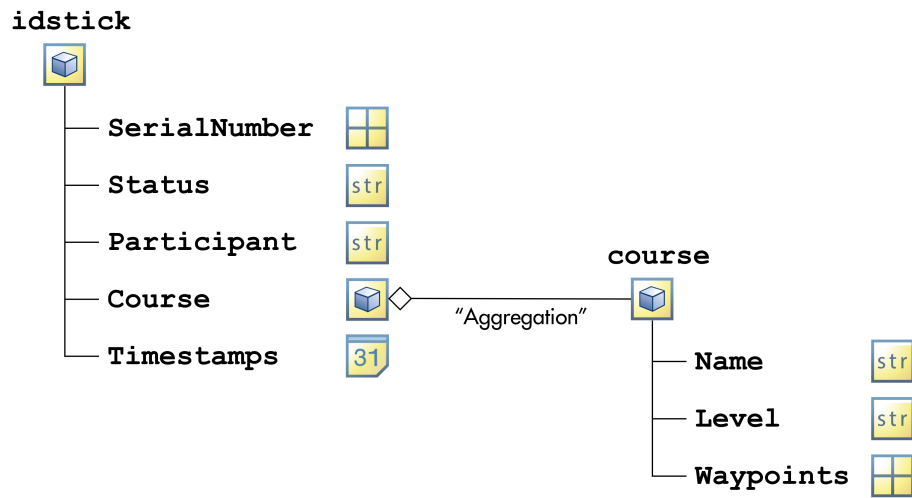
您可以通过将值赋给 `object.Property` 来修改属性值。

```
obj.PropertyName = newvalue
```

对象作为属性

您现在有一个基本 `course` 类。接下来，您需要创建一个类来表示 ID 棒。该类的属性之一将保留参赛人员正在行进的路线。这意味着 `idstick` 对象的属性之一应包含一个 `course` 对象。这可能吗？一个对象能包含另一个对象吗？

是的！任何属性都可以保留各种大小或类型的数据，甚至其他自定义对象。



当您构建具有多个类的应用程序时，通常会像这样在对象中包含对象。在 OOP 世界中，这种关系（“classA 包含 classB”）称为关联。以上这种关联（例如路线可以独立于 ID 棒而存在，但 ID 棒包含一个路线）称为聚合。